System Verification & Architecture Report

UART-Controlled ALU and Register File System

Digital IC Design Diploma by Ali Eltemsah – Final Project

Author: Ahmed Nageeb

October 6, 2026

Contents

1 System Overview **2**

2 System Controller (SYS_CTRL) Architecture **3**
2.1 Finite State Machine Operations 4
2.2 Command Decoding & Execution 5
2.3 Memory Management 5
2.4 Operational Flow 5

3 Verification Test Cases **6**
3.1 Phase 1: Register File Write Verification (Address 0x05) 7
3.2 Phase 2: Register File Write Verification (Address 0x06) 8
3.3 Phase 3: Register File Read Verification (Address 0x05) 9
3.4 Phase 4: Register File Read Verification (Address 0x06) 10
3.5 Phase 5: ALU Operation With Operands (Addition) 11
3.6 Phase 6: ALU Operation Without Operands (Subtraction) 12
3.7 Phase 7: ALU Multiplication (16-bit Output Stream - Lower Byte) . . . 13
3.8 Phase 8: ALU Multiplication (16-bit Output Stream - Upper Byte) . . . 14
3.9 Phase 9: FIFO Stall and Backpressure Recovery 15

4 Debugging & Issue Resolution **17**
4.1 Bug 1: SYS_CTRL Combinational Amnesia 17
4.2 Bug 2: The Multiplication Mathematical Illusion 17
4.3 Bug 3: UART Receiver Synchronous Pipeline Race Condition 17

1 System Overview

This report documents the architectural design, operational states, and functional verification of the system architecture. The system integrates 10 primary functional blocks separated across two distinct clock domains.

Clock Domain 1 operates on a 100 MHz reference clock and encompasses the Register File, ALU, Clock Gating, and the System Controller (`SYS_CTRL`). Clock Domain 2 operates on a 3.6864 MHz UART clock (`UART_CLK`) and manages the UART Transmitter (`UART_TX`), UART Receiver (`UART_RX`), Pulse Generator, and Clock Dividers.

Data boundaries and cross-domain synchronization are safely managed via asynchronous FIFOs, data synchronizers, and reset synchronizers. The system receives operational commands from a master testbench via the `UART_RX` interface, executes the requested arithmetic or storage functions, and transmits the resultant data back through the `UART_TX` interface.

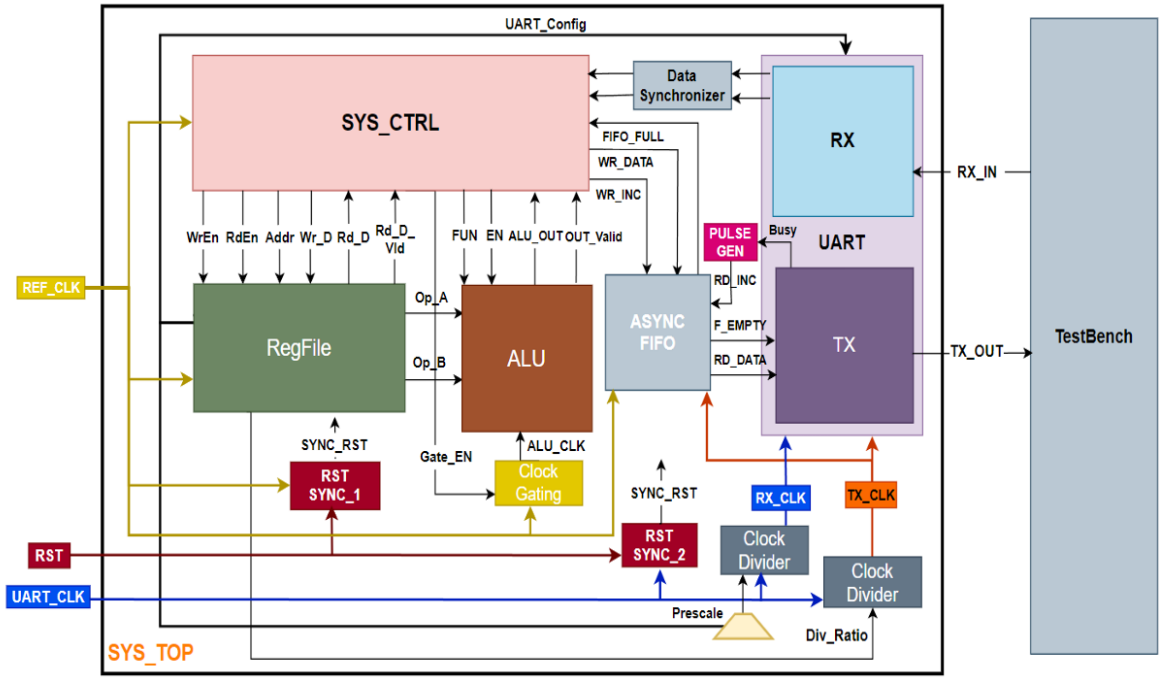


Figure 1: System Overall Architecture and Block Diagram

2 System Controller (SYS_CTRL) Architecture

The core orchestrator of Clock Domain 1 is the `SYS_CTRL` Finite State Machine (FSM). This module sits at the center of the datapath, interfacing directly with the ALU, Register File, UART RX bus, and the Asynchronous TX FIFO.

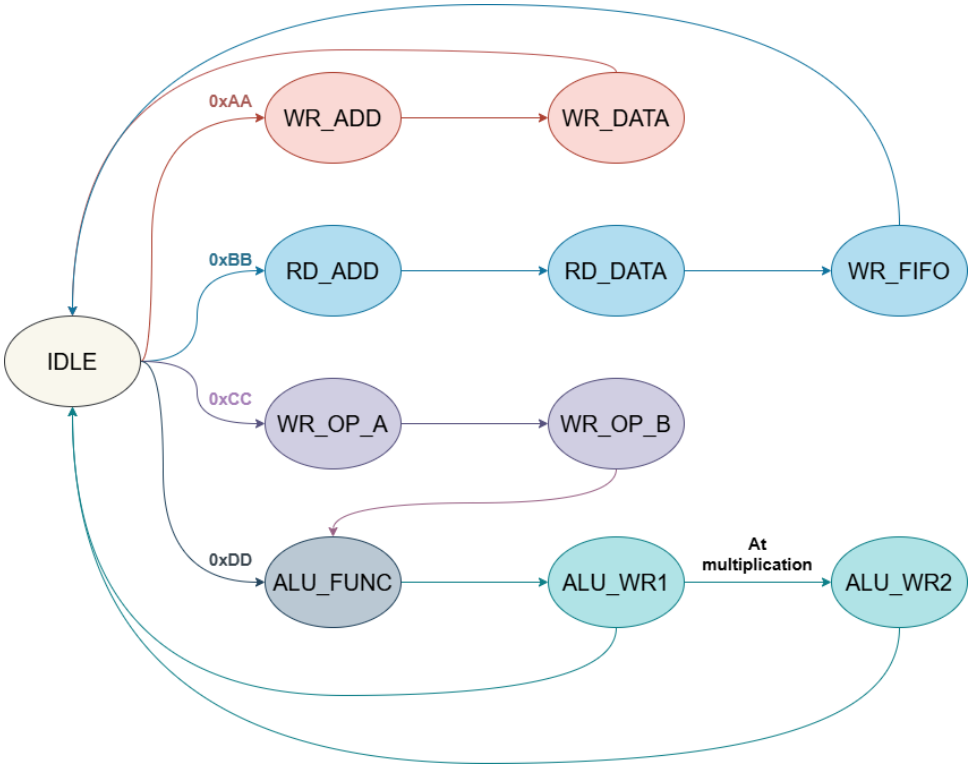


Figure 2: SYS_CTRL Finite State Machine (FSM) State Diagram

2.1 Finite State Machine Operations

The SYS_CTRL module implements an 11-state Finite State Machine to govern exact system timing and operations. The states behave strictly as follows:

- **IDLE (4'b0000):** The system rests with default outputs disabled. It evaluates UART_RX_DATA upon a valid pulse (UART_RX_VLD) to decode command hexes (0xAA, 0xBB, 0xCC, 0xDD) and branch to the respective execution paths.
- **WR_ADD (4'b0001):** Entered for Register Write operations. It waits for the next valid UART frame and securely captures the target address into the sequential register memory.
- **WR_DATA (4'b0011):** The FSM asserts RF_WrEn, points RF_Address to the stored register value, and drives RF_WrData with the incoming UART_RX_DATA. It completes the write and returns to IDLE upon the valid pulse.
- **RD_ADD (4'b0010):** Entered for Register Read operations. Upon receiving the address frame, it immediately asserts RF_RdEn and points RF_Address directly to the UART_RX_DATA, while simultaneously backing it up in register.
- **RD_DATA (4'b0110):** The state maintains RF_RdEn using the stored register address. It halts state progression until the Register File acknowledges with RF_RdData_VLD.
- **WR_FIFO (4'b0111):** While keeping read enable asserted, the state evaluates FIFO_FULL. Once space is available, it pulls RF_RdData into UART_TX_DATA and triggers UART_TX_VLD to transmit the byte.
- **WR_OP_A (4'b0101):** Part of the ALU with operands sequence. Upon a valid frame, it hardcodes RF_Address to 0 (REG0) and asserts RF_WrEn to load Operand A.
- **WR_OP_B (4'b0100):** Sequentially follows A. Upon a valid frame, it hardcodes RF_Address to 1 (REG1) and asserts RF_WrEn to load Operand B.
- **ALU_FUNC (4'b1100):** Enables clock gating via CLKG_EN. Upon receiving the final UART frame, it routes the byte directly to ALU_FUN, triggers ALU_EN, and memorizes the operation code in register.
- **ALU_WR1 (4'b1101):** Holds the ALU_FUN, ALU_EN, and CLKG_EN active. Waits for both ALU_OUT_VLD and !FIFO_FULL to transmit the lower 8 bits (ALU_OUT[7:0]). If the memorized instruction is multiplication (8'b00000010), it branches to ALU_WR2; otherwise, it returns to IDLE.
- **ALU_WR2 (4'b1111):** Maintains ALU enablement and waits for FIFO backpressure to clear before transmitting the upper 8 bits (ALU_OUT[15:8]). Returns to IDLE once transmitted.

2.2 Command Decoding & Execution

The FSM is responsible for evaluating serialized hex command frames received from the UART interface and directing the appropriate sub-modules. The controller supports four primary command structures:

- **Register File Write (`0xAA`):** A 3-frame command consisting of the `0xAA` instruction, the target address, and the data byte.
- **Register File Read (`0xBB`):** A 2-frame command consisting of the `0xBB` instruction followed by the target address.
- **ALU Operation with Operands (`0xCC`):** A 4-frame command consisting of the `0xCC` instruction, Operand A, Operand B, and the ALU function code.
- **ALU Operation without Operands (`0xDD`):** A 2-frame command consisting of the `0xDD` instruction and the ALU function code.

2.3 Memory Management

During ALU operations, the `SYS_CTRL` interacts with specific reserved addresses within the 8x16 Register File.

- Operand A is stored in `REG0` at Address `0x0`.
- Operand B is stored in `REG1` at Address `0x1`.
- Address `0x2` is reserved for UART Configuration (Parity Enable, Parity Type, and Prescaler).
- Address `0x3` is reserved for the Clock Division Ratio.
- Addresses `0x4` through `0x15` are available for standard read/write operations.

2.4 Operational Flow

Upon receiving a valid start frame, the `SYS_CTRL` transitions from `IDLE` to the corresponding operational branch. For ALU commands, it activates the ALU clock via the Clock Gating block. The FSM holds its evaluation states until the ALU asserts `OUT_Valid`. Upon completion, the `SYS_CTRL` pushes the resultant data to the Asynchronous FIFO by asserting `WR_INC` and `WR_DATA`.

3 Verification Test Cases

The verification plan executes nine distinct test phases via an automated testbench master that transmits serialized hex commands, evaluates the system's asynchronous responses, and validates backpressure stall recovery.
Global Configuration Note: All verification test cases are executed using a configured **UART_RX Clock Prescale Value = 32** and **Even Parity checking enabled**.

3.1 Phase 1: Register File Write Verification (Address 0x05)

Objective: Verify that the system correctly decodes the Write Command (`0xAA`) to route and populate the Register File at the first target address.
Stimulus:

- Write: `0xAA` → Addr `0x05` → Data `0x11`

Expected Behavior: The system updates the internal register at address `0x05` with the data `0x11`. No UART transmission is expected back to the master.
Waveform Snapshot:

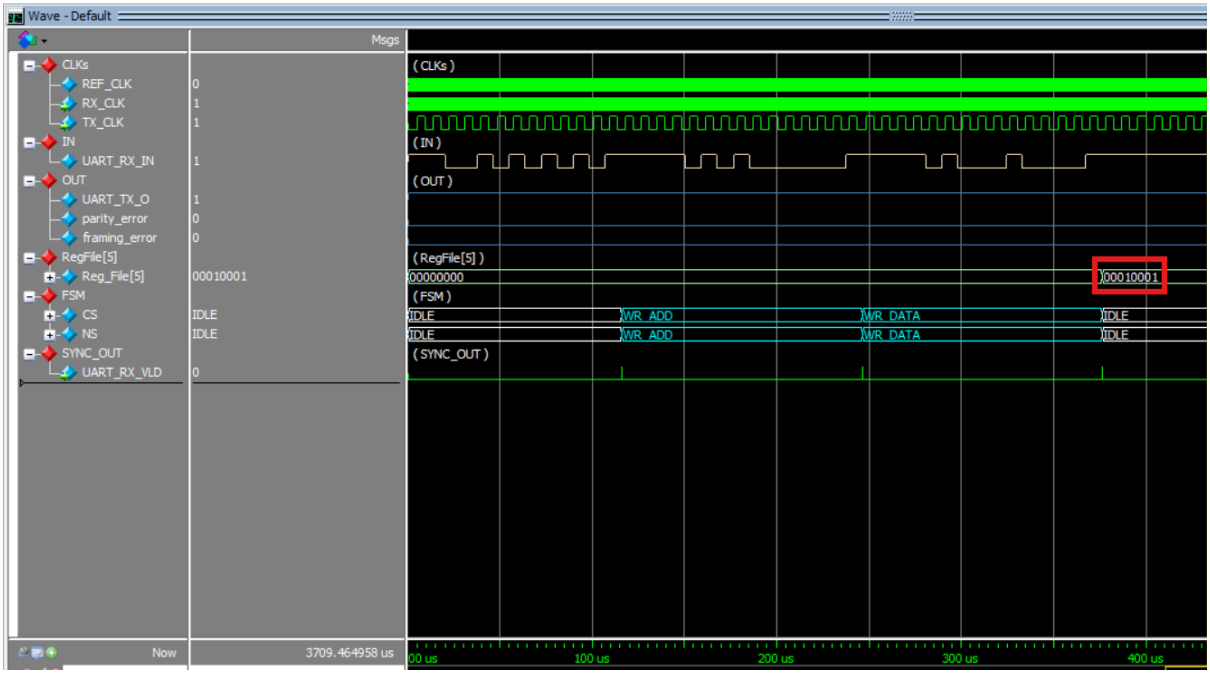


Figure 3: Waveform Snapshot for Register File Write Verification (Address 0x05)

3.2 Phase 2: Register File Write Verification (Address 0x06)

Objective: Verify that the system correctly decodes a consecutive Write Command (`0xAA`) to route and populate the Register File at a second target address.
Stimulus:

- Write: `0xAA` → Addr `0x06` → Data `0xA5`

Expected Behavior: The system updates the internal register at address `0x06` with the data `0xA5`. No UART transmission is expected back to the master.
Waveform Snapshot:

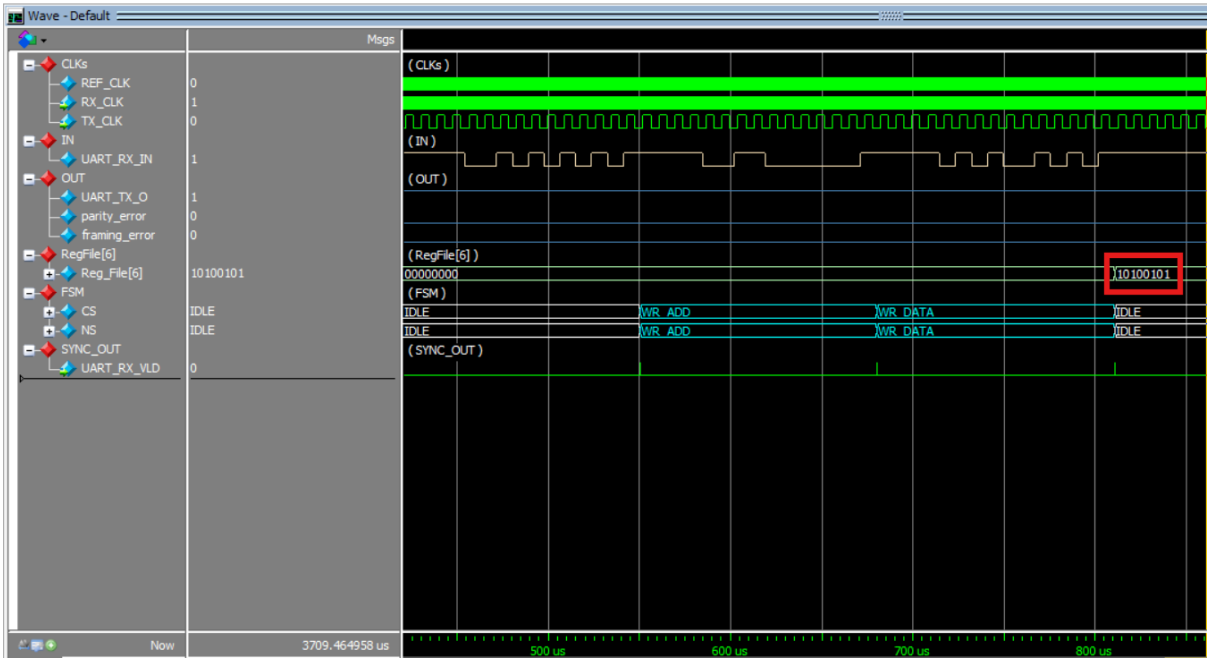


Figure 4: Waveform Snapshot for Register File Write Verification (Address 0x06)

3.3 Phase 3: Register File Read Verification (Address 0x05)

Objective: Verify that the system correctly decodes the Read Command (`0xBB`) to fetch target data from the first stored address and transmit it back via `UART_TX`.
Stimulus:

- Read: `0xBB` → Addr `0x05`

Expected Behavior: The system successfully fetches the previously stored value from Phase 1 and transmits `0x11`.
Transcript Output:

```
[PASS] REG_RD_0x05 | Exp: 0x11 | Got: 0x11
```

Waveform Snapshot:

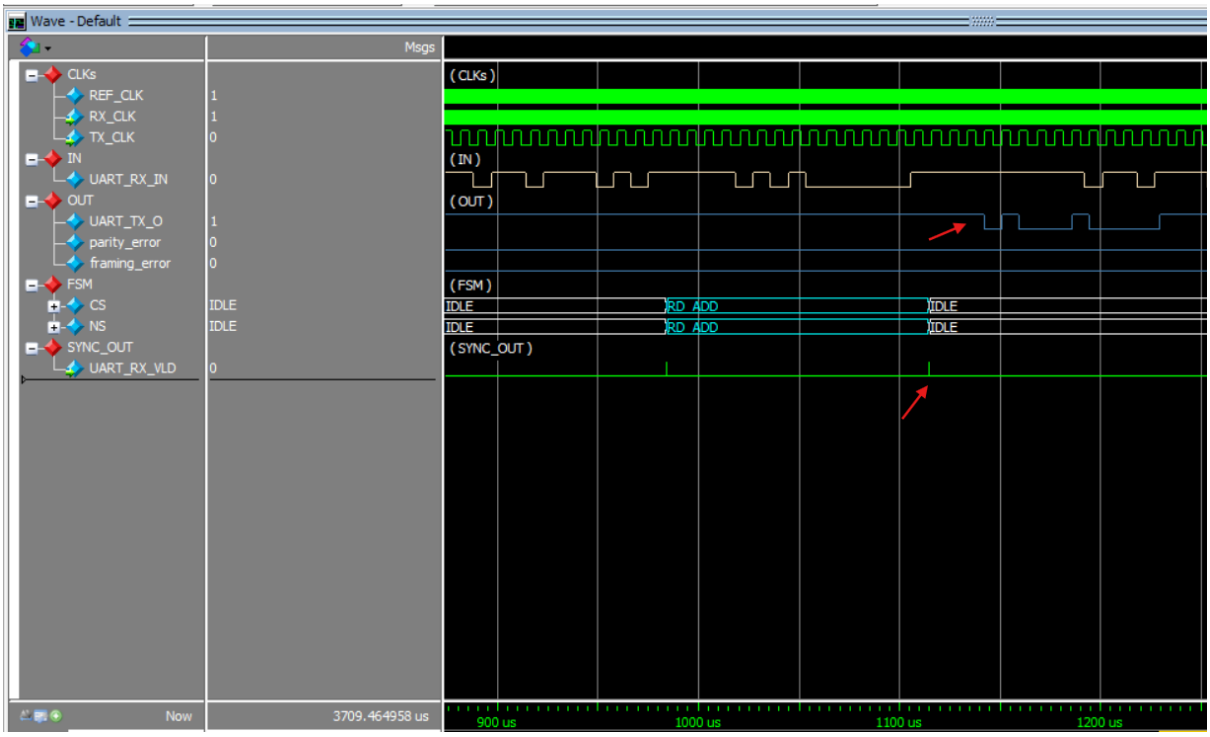


Figure 5: Waveform Snapshot for Register File Read Verification (Address 0x05)

3.4 Phase 4: Register File Read Verification (Address 0x06)

Objective: Verify that the system correctly decodes the Read Command (`0xBB`) to fetch target data from the second stored address and transmit it back via `UART_TX`.
Stimulus:

- Read: `0xBB` → Addr `0x06`

Expected Behavior: The system successfully fetches the previously stored value from Phase 2 and transmits `0xA5`.
Transcript Output:

```
[PASS] REG_RD_0x06 | Exp: 0xa5 | Got: 0xa5
```

Waveform Snapshot:



Figure 6: Waveform Snapshot for Register File Read Verification (Address 0x06)

3.5 Phase 5: ALU Operation With Operands (Addition)

Objective: Verify the execution of an ALU command (`0xCC`) providing two new operands and an opcode.
Stimulus:

- Frame 0 (CMD): `0xCC`
- Frame 1 (OP_A): `0x08`
- Frame 2 (OP_B): `0x04`
- Frame 3 (FUN): `0x00` (Addition)

Expected Behavior: The ALU computes $0x08 + 0x04 = 0x0C$, and the controller transmits the result byte.
Transcript Output:

```
[PASS] ALU_ADD_8+4 | Exp: 0x0c | Got: 0x0c
```

Waveform Snapshot:

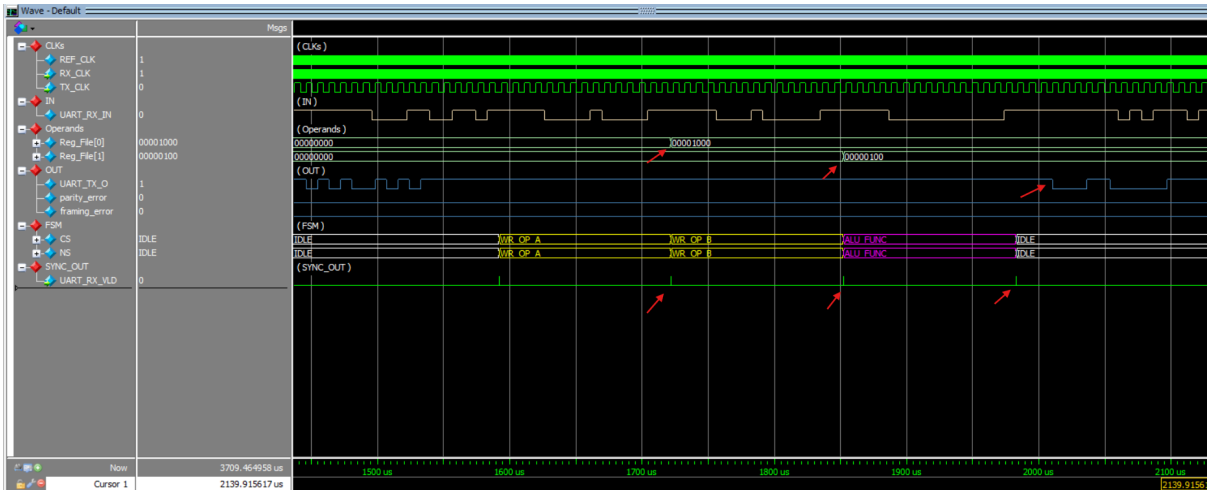


Figure 7: Waveform Snapshot for ALU Addition Operation

3.6 Phase 6: ALU Operation Without Operands (Subtraction)

Objective: Verify that an ALU command (`0xDD`) correctly bypasses operand initialization and re-uses previously stored operands (A=`0x08`, B=`0x04`) for a new operation.
Stimulus:

- Frame 0 (CMD): `0xDD`
- Frame 1 (FUN): `0x01` (Subtraction)

Expected Behavior: The ALU computes $0x08 - 0x04 = 0x04$, and transmits the result byte.
Transcript Output:

```
[PASS] ALU_SUB_8-4 | Exp: 0x04 | Got: 0x04
```

Waveform Snapshot:

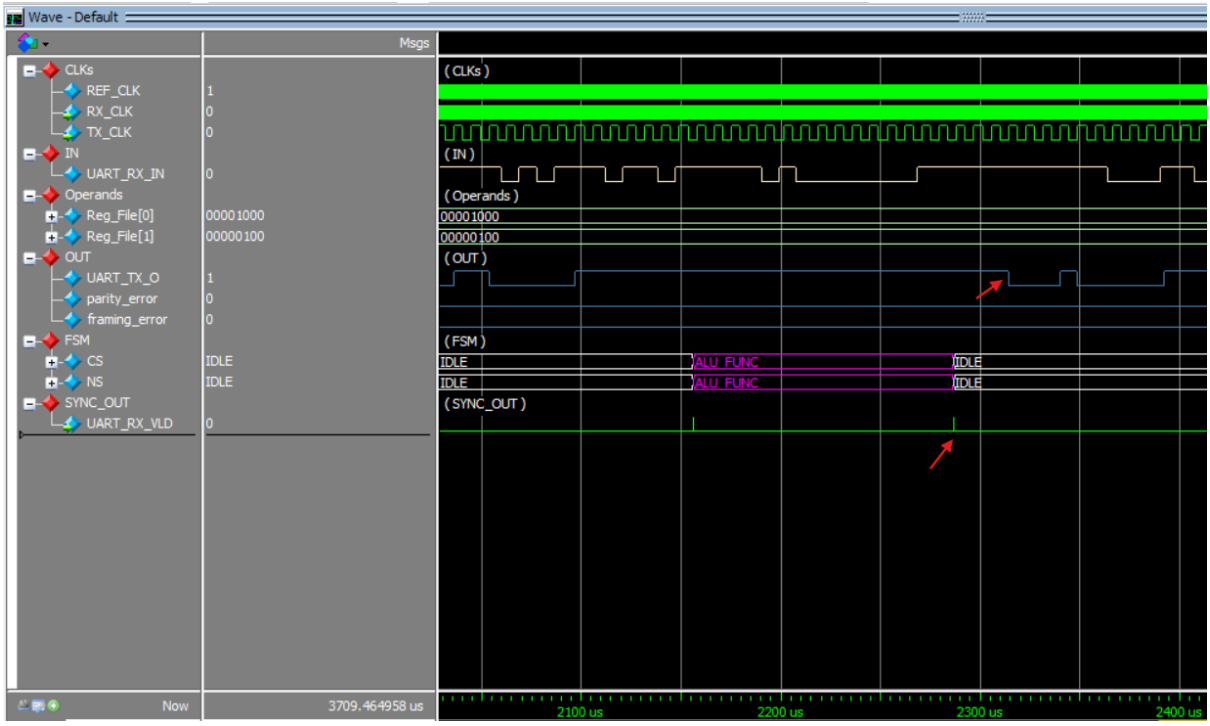


Figure 8: Waveform Snapshot for ALU Subtraction Operation

3.7 Phase 7: ALU Multiplication (16-bit Output Stream - Lower Byte)

Objective: Verify that operations yielding 16-bit results trigger the `SYS_CTRL` to split the data and transmit the first sequential UART frame (Lower Byte) without dropping states.
Stimulus:

- Frame 0 (CMD): `0xCC`
- Frame 1 (OP_A): `0x10`
- Frame 2 (OP_B): `0x10`
- Frame 3 (FUN): `0x02` (Multiplication)

Expected Behavior: $0x10 \times 0x10 = 0x0100$. The system must transmit `0x00` (Lower).
Transcript Output:

```
[PASS] ALU_MULT_LOWER | Exp: 0x00 | Got: 0x00
```

Waveform Snapshot:

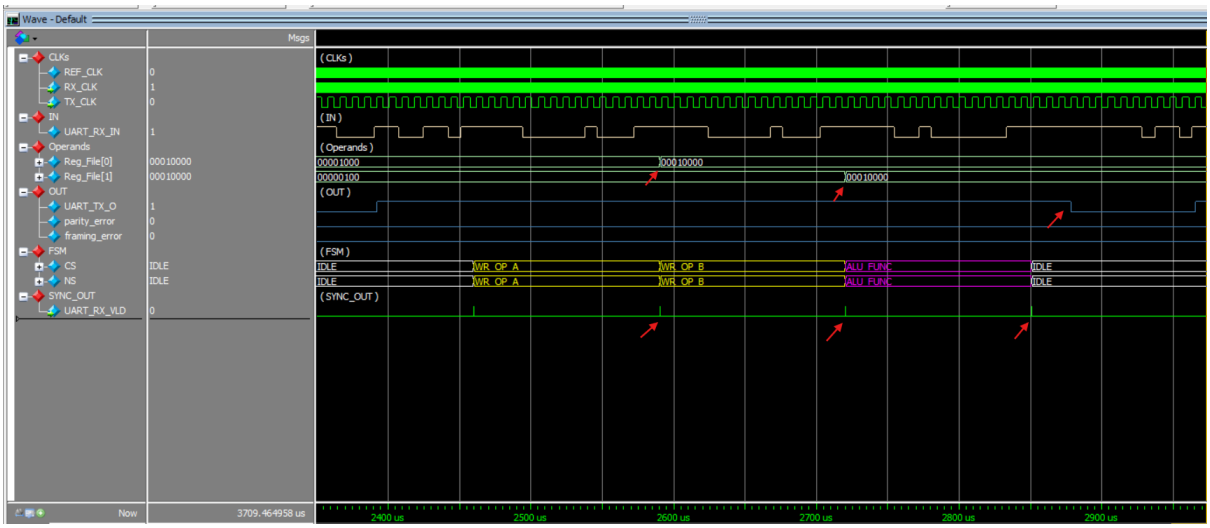


Figure 9: Waveform Snapshot for ALU Multiplication (Lower Byte)

3.8 Phase 8: ALU Multiplication (16-bit Output Stream - Upper Byte)

Objective: Verify that operations yielding 16-bit results trigger the `SYS_CTRL` to transmit the second sequential UART frame (Upper Byte) without dropping states.
Stimulus:

- (Automated sequential execution following Phase 7)

Expected Behavior: Following the lower byte, the system must transmit `0x01` (Upper).
Transcript Output:

```
[PASS] ALU_MULT_UPPER | Exp: 0x01 | Got: 0x01
```

Waveform Snapshot:

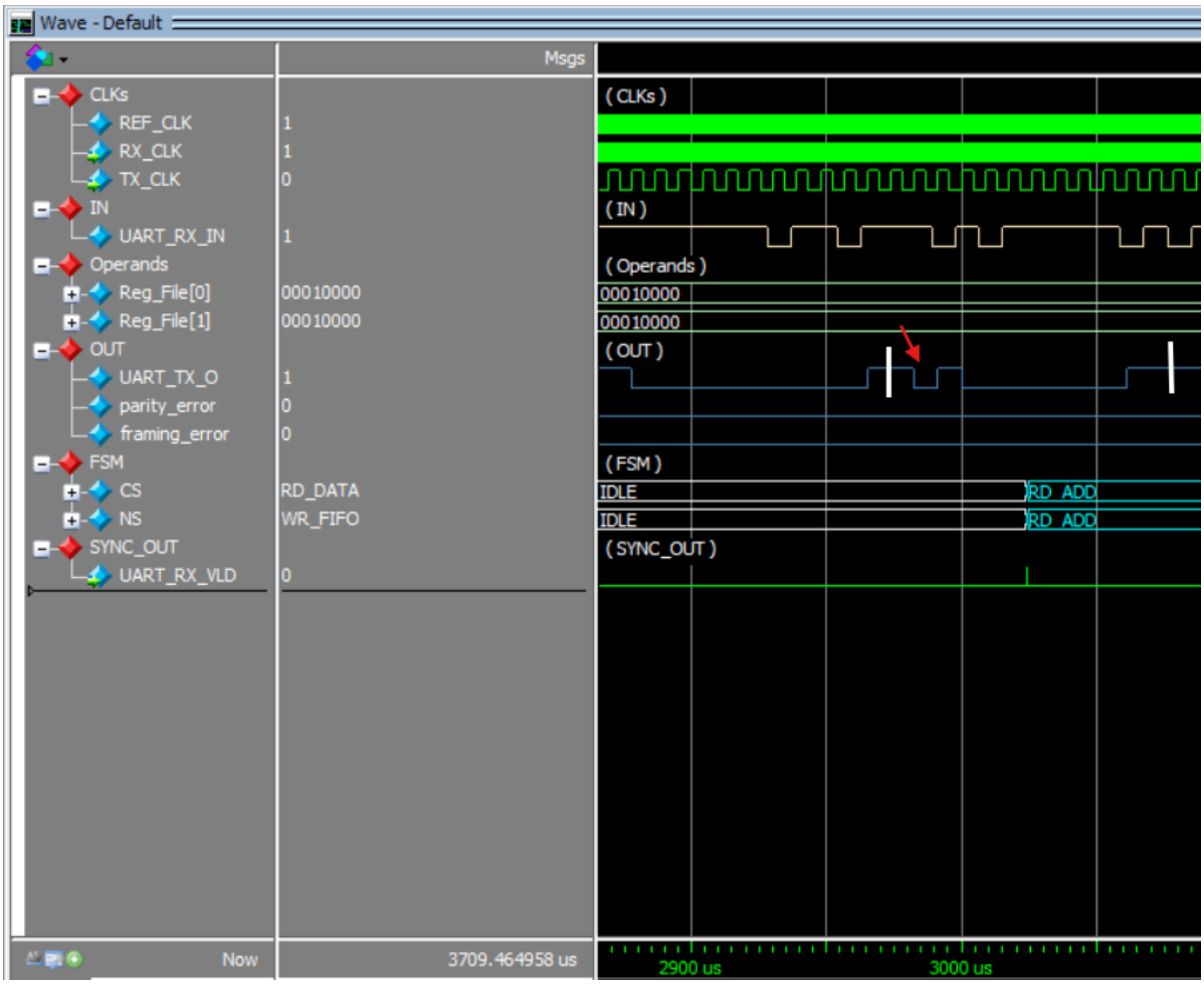


Figure 10: Waveform Snapshot for ALU Multiplication (Upper Byte)

3.9 Phase 9: FIFO Stall and Backpressure Recovery

Objective: Verify that the `SYS_CTRL` FSM correctly halts state progression when the asynchronous TX FIFO asserts a full condition, and cleanly resumes operation without data loss once the stall is cleared.
Stimulus:

- Assert `FIFO_FULL = 1'b1` via testbench force.
- Issue Read Command (`0xBB`) for Address `0x05`.
- Wait 20 bit-periods.
- Release `FIFO_FULL = 1'b0`.

Expected Behavior: The FSM enters the read sequence but waits indefinitely in the FIFO write state. Once `FIFO_FULL` drops, the controller resumes and transmits `0x11`.
Transcript Output:

```
[PASS] FIFO_STALL_RECOVERY | Exp: 0x11 | Got: 0x11
```

Waveform Snapshot:

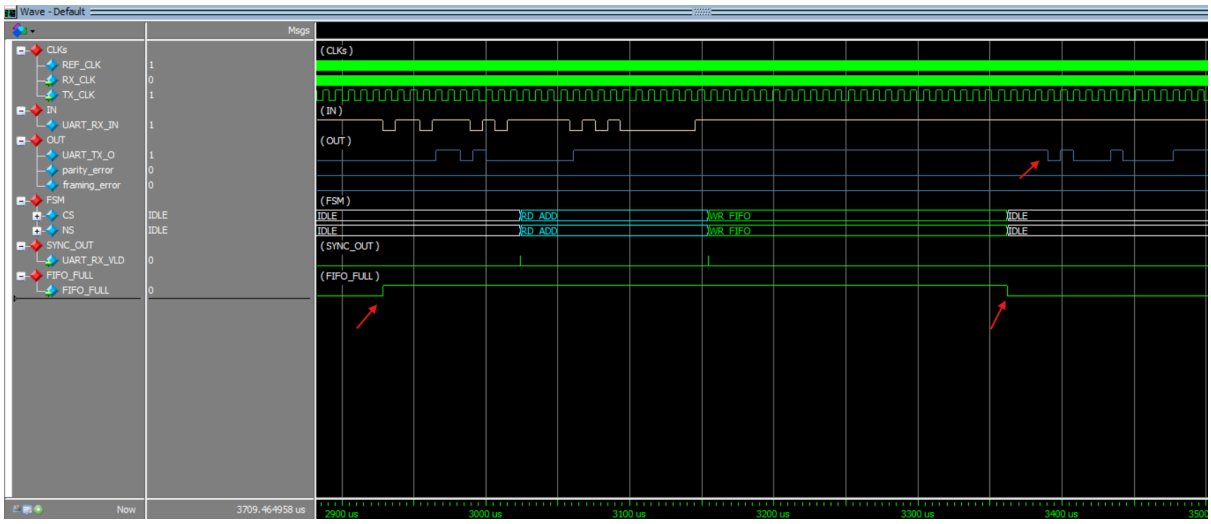


Figure 11: Waveform Snapshot for FIFO Stall and Backpressure Recovery

Notice that there is actually a shift between Current state and Next state but it does not appear in the zoomed out waveform due to the high frequency reference clock "REF_CLK" that runs the system and notice that at multiplication operation the FSM actually goes to ALU_WR1 and ALU_WR2 states but they do not appear in the zoomed out waveform due to the high frequency reference clock "REF_CLK" also.

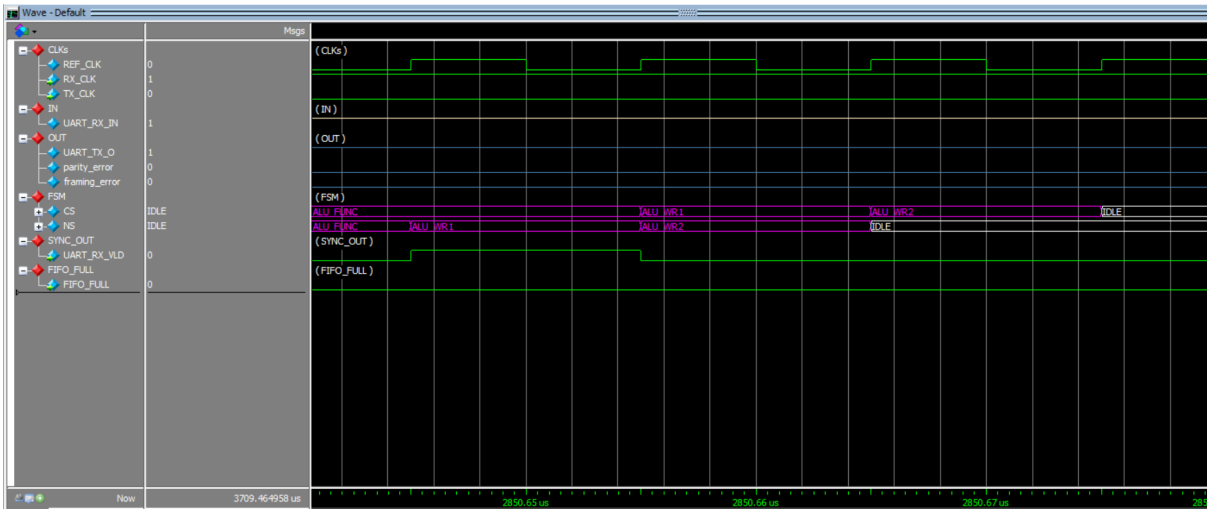


Figure 12: Detailed Zoomed-in Waveform Snapshot

4 Debugging & Issue Resolution

During the integration phase, several critical logic flaws were identified. This section details the symptoms, root causes, and applied fixes for the most notable issues.

4.1 Bug 1: SYS_CTRL Combinational Amnesia

Symptom: The system failed to fetch data during Read operations and improperly overwrote ALU Operand A during Write operations.
Root Cause: The `register` variable in `SYS_CTRL.v` was modeled entirely inside a combinational `always @(*)` block with a default assignment of `0`. Upon transitioning states, the `register` immediately evaluated back to `0`, losing the stored address.
Resolution: The `register` variable was migrated to the sequential `always @(posedge CLK)` block, turning it into true flip-flop memory that retains values perfectly across state transitions.

4.2 Bug 2: The Multiplication Mathematical Illusion

Symptom: Testing `0x02 * 0xFF` yielded `0xFE` as the lower byte, but erroneously output `0x01` as the upper byte due to a coincidental arithmetic masking of the issue.
Root Cause: While the `ALU_WR1` state kept `ALU_EN = 1`, it failed to explicitly assert `ALU_FUN = register`. The combinational ALU logic defaulted to Addition (`0x00`), computing `0x02 + 0xFF = 0x0101`. The controller transmitted the lower byte of the multiplication (`0xFE`), but grabbed the upper byte of the addition (`0x01`).
Resolution: The `ALU_FUN` assignment was explicitly maintained during the `ALU_WR1` and `ALU_WR2` states to lock the arithmetic logic into multiplication.

4.3 Bug 3: UART Receiver Synchronous Pipeline Race Condition

Symptom: The FSM ignored the very first UART valid tick, resulting in state transitions operating "shifted by one byte." This caused multi-byte commands, specifically the 4-byte Multiplication operation, to hang permanently in the `ALU_FUNC` state while the final byte (`0x02`) remained trapped inside the receiver buffer.
Root Cause: A classic same-edge synchronous race condition. The `DATA_VALID` signal was used simultaneously as a clock enable for the deserializer to update its `P_DATA` register, and as a clock enable for the `SYS_CTRL` to read `P_DATA`. Because both operations fired on the exact same clock edge via non-blocking assignments, the FSM perpetually sampled the *old* data a fraction of a nanosecond before the flip-flops updated with the new byte.
Resolution: The `RX_FSM` module was refactored to implement staggered enables. An early `DESER_STORE` signal was routed to the deserializer to latch the data natively. The top-level `DATA_VALID` signal driving the controller was delayed by one internal flip-flop stage, staggering the pipeline and allowing `P_DATA` to settle completely before the system read it.